

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**ĐÁNH LỪA MÔ HÌNH HỌC SÂU
VỚI TẦN CÔNG MỘT PIXEL**

BÁO CÁO CUỐI KỲ

Môn học: Các vấn đề hiện đại trong Công nghệ thông tin

Mã lớp học phần: INT3507 7

Giảng viên: TS. Nguyễn Đức Anh

Nhóm số 7

Nguyễn Hùng Dũng 22021133

Ngô Thành Minh 22021131

Vũ Văn Huy 22021202

Nguyễn Việt Anh 22021162

Hà Nội, 2024

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**ĐÁNH LỪA MÔ HÌNH HỌC SÂU
VỚI TẦN CÔNG MỘT PIXEL**

BÁO CÁO CUỐI KỲ

Môn học: Các vấn đề hiện đại trong Công nghệ thông tin

Mã lớp học phần: INT3507 7

Giảng viên: TS. Nguyễn Đức Anh

Nhóm số 7

Nguyễn Hùng Dũng 22021133

Ngô Thành Minh 22021131

Vũ Văn Huy 22021202

Nguyễn Việt Anh 22021162

Hà Nội, 2024

Phân công công việc

Tên thành viên	Mã sinh viên	Phân công công việc	Đóng góp
Nguyễn Hùng Dũng	22021133	Nhóm trưởng; Thực hiện và viết báo cáo kết quả các thí nghiệm; Viết báo cáo phần giới thiệu bài toán, mô hình hóa bài toán và phụ lục	25%
Ngô Thành Minh	22021131	Tìm hiểu và viết báo cáo mô tả chi tiết cách hoạt động của tiền hóa vi phân và phương pháp tấn công một pixel	25%
Vũ Văn Huy	22021202	Tìm hiểu và viết báo cáo về các thực nghiệm của các tác giả	25%
Nguyễn Việt Anh	22021162	Tìm hiểu và mô tả các nghiên cứu liên quan trong bài toán tấn công đối kháng	25%

Bảng 1: Danh sách thành viên và phân công công việc

Tóm tắt

Nhiều nghiên cứu gần đây đã chỉ ra rằng các mô hình học sâu nhận diện hình ảnh có thể bị đánh lừa và nhận diện sai hoàn toàn khi ảnh đầu vào bị thêm một lượng nhiễu rất nhỏ. Việc đánh lừa mô hình nhận diện hình ảnh bằng hình thức thêm nhiễu vào ảnh như vậy gọi là tấn công đối kháng.

Trong báo cáo này, chúng em giới thiệu một phương pháp tấn công đối kháng mang tên Tấn công một pixel, được đề xuất bởi các tác giả Jiawei Su, Danilo Vasconcellos Vargas và Kouichi Sakurai vào năm 2019 trong bài báo có tên "One Pixel Attack for Fooling Deep Neural Networks" [15]. Phương pháp này cố gắng đánh lừa các mô hình nhận diện hình ảnh trong trường hợp vô cùng hạn chế: chỉ được phép gây nhiễu trên một pixel trong ảnh đầu vào. Phương pháp này hoạt động dựa trên thuật toán Tối ưu hóa vi phân, yêu cầu rất ít thông tin của mô hình mục tiêu và có thể đánh lừa nhiều loại mô hình khác nhau.

Từ khoá: nhận diện hình ảnh, tấn công đối kháng, học sâu, tối ưu hóa, tấn công hộp đen

Lời cam đoan

Chúng em xin cam đoan rằng toàn bộ nội dung trong bài báo cáo đều do nhóm tự tìm hiểu, nghiên cứu và hoàn thành dựa trên sự hướng dẫn tận tình của giảng viên. Chúng em cam kết không sao chép nội dung hay sử dụng các phần mềm trí tuệ nhân tạo như ChatGPT, Gemini,... Tất cả các tài liệu tham khảo sử dụng trong báo cáo đều được trích dẫn nguồn gốc đầy đủ và nêu rõ trong mục Tài liệu tham khảo. Nếu phát hiện bất kì sự gian lận nào, chúng em xin chịu trách nhiệm trước giảng viên hướng dẫn, trước khoa và nhà trường.

Thành viên 1

Thành viên 2

Thành viên 3

Thành viên 4

Mục lục

Phân công công việc	iii
Tóm tắt	iv
Lời cam đoan	v
Mục lục	vi
Danh sách các hình	vii
Danh sách các bảng	viii
1 Giới thiệu	1
1.1 Giới thiệu về tấn công đối kháng	1
1.2 Giới thiệu về tấn công một pixel	1
2 Các nghiên cứu liên quan	4
2.1 Phân loại các phương pháp tấn công đối kháng	4
2.1.1 Dựa trên quyền truy cập thông tin	4
2.1.2 Dựa trên mục tiêu tấn công	4
2.1.3 Dựa trên loại nhiễu thêm vào	5
2.2 Fast Gradient Sign Method (FGSM)	6
2.2.1 Mục tiêu	6
2.2.2 Ưu - nhược điểm	6
2.2.3 Ứng dụng	6
2.3 Projected Gradient Descent (PGD)	7
2.3.1 Mục tiêu	7
2.3.2 Ưu - nhược điểm	7
2.3.3 Ứng dụng	7
2.4 Carlini & Wagner (C&W) Attack	7
2.4.1 Mục tiêu	7
2.4.2 Ưu - nhược điểm	8
2.5 Zero-Order Optimization (ZOO) Attack	8
2.5.1 Mục tiêu	8
2.5.2 Ưu - nhược điểm	8
3 Tấn công một pixel	9
3.1 Mô hình hóa bài toán	9

3.2	Tiến hóa vi phân	9
3.2.1	Nguyên lý hoạt động của tiến hóa vi phân	10
3.2.2	Một vài ưu điểm của tiến hóa vi phân	10
3.3	Tần công một pixel	11
4	Thực nghiệm và Kết quả	13
4.1	Thiết lập thí nghiệm	13
4.2	Kết quả thực nghiệm	14
A	Phụ lục	17
A.1	Sử dụng phân phối chuẩn ở bước khởi tạo quần thể	17
A.2	Tạo đột biến theo trung bình cộng của ba cá thể	18
A.3	So sánh với việc thử thay đổi từng pixel	18
A.4	Nhận xét	19
	Tài liệu tham khảo	20

Danh sách các hình

1.1	Ảnh tần công đối kháng khiến mô hình nhận diện con lợn thành cái máy bay.	1
1.2	Tần công một pixel trên tập CIFAR10, đánh lừa 3 mô hình học sâu: All convolutional network, Network in network và VGG. Chữ màu đen là nhãn đúng, còn chữ màu xanh là nhãn do mô hình dự đoán ra cùng với độ tự tin của mô hình vào nhãn đó	2
1.3	Tần công một pixel trên tập ImageNet	2
4.1	Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tần công 1 pixel.	15
4.2	Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tần công 3 pixel.	15
4.3	Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tần công 5 pixel.	15

Danh sách các bảng

1	Danh sách thành viên và phân công công việc	iii
4.1	Tỉ lệ thành công khi tấn công một pixel trên 4 mô hình khác nhau. 3 mô hình AllConv, NiN, VGG16 được huấn luyện trên tập CIFAR10, và mô hình BVLC được huấn luyện trên ImageNet.	14
4.2	Tỉ lệ thành công khi tấn công 3 pixel trên mạng nơ-ron All convolution network (AllConv) và tấn công 5 pixel trên mạng nơ-ron Network in Network (NiN).	14
4.3	Đánh giá chi phí thực hiện một cuộc tấn công 1 pixel trên bốn loại mạng nơ-ron AllConv, NiN, VGG16, BVLC.	16
A.1	Kết quả thí nghiệm so sánh hai cách chọn điểm ảnh bị thay đổi	18
A.2	Kết quả thí nghiệm so sánh hai cách tạo đột biến	18

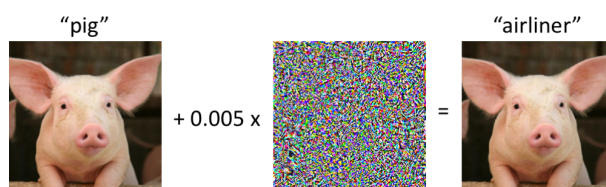
Chương 1

Giới thiệu

1.1 Giới thiệu về tấn công đối kháng

Trong lĩnh vực nhận diện hình ảnh, các phương pháp sử dụng mạng nơ-ron sâu (DNN) đã chứng minh hiệu quả vượt trội so với các phương pháp truyền thống, đạt được kết quả ngang bằng, thậm chí là tốt hơn con người [16].

Tuy nhiên, một vài nghiên cứu đã chỉ ra các nhiễu nhân tạo trên ảnh có thể dễ dàng khiến các DNN nhận diện sai. Quá trình tạo ra những nhiễu như vậy nhằm đánh lừa mô hình gọi là tấn công đối kháng, và ảnh chứa các nhiễu đó gọi là ảnh tấn công đối kháng [7] [11]. Ý tưởng cơ bản của tấn công đối kháng là thêm một lượng nhiễu rất nhỏ đã được tính toán kỹ vào hình ảnh, sao cho mắt người không nhận ra được. Những thay đổi này có thể làm mô hình học sâu phân loại hình ảnh sang các nhãn hoàn toàn khác.



Hình 1.1: Ảnh tấn công đối kháng khiến mô hình nhận diện con lợn thành cái máy bay.

Tuy nhiên, đa số các phương pháp tấn công trước đã không cân nhắc các trường hợp mà sự thay đổi bị giới hạn, và lượng điểm ảnh bị thay đổi thường rất lớn. Việc nghiên cứu tấn công đối kháng trong các tình huống bị giới hạn có thể mở ra những góc nhìn mới về tính chất hình học và cách hoạt động của DNN trong không gian nhiều chiều [6].

1.2 Giới thiệu về tấn công một pixel

Trong bài báo [15], các tác giả đề xuất một phương pháp tấn công hợp đen đối với DNN trong bối cảnh mà xác suất các nhãn là thông tin duy nhất về mô hình có thể truy cập, và chỉ được phép thay đổi một điểm ảnh duy nhất. Phương pháp này gọi là Tấn công một pixel [15], hoạt động dựa trên thuật toán tối ưu hóa Tiến hóa vi phân [14] để tìm ra pixel cần gây nhiễu và lượng nhiễu cần thiết để đánh lừa mô hình thành công.



Hình 1.2: Tấn công một pixel trên tập CIFAR10, đánh lừa 3 mô hình học sâu: All convolutional network, Network in network và VGG. Chữ màu đen là nhãn đúng, còn chữ màu xanh là nhãn do mô hình dự đoán ra cùng với độ tự tin của mô hình vào nhãn đó

Phương pháp này có các lợi thế sau:

- **Tính hiệu quả:** Trên tập dữ liệu Kaggle CIFAR-10, tấn công không mục tiêu bằng cách thay đổi chỉ một điểm ảnh trên 3 mô hình DNN khác nhau đạt tỉ lệ thành công 68.71%, 71.66% và 63.53%.
- **Là tấn công hộp đen:** Chỉ cần biết xác suất các nhãn mà mô hình đưa ra, không cần thông tin về kiến trúc mô hình. Phương pháp này cũng rất đơn giản vì không cần sử dụng hàm mục tiêu phức tạp, mà chỉ tập trung vào tăng xác suất của nhãn mục tiêu.
- **Tính linh hoạt:** Có thể tấn công nhiều loại mô hình DNN, kể cả các mô hình không khả vi hoặc các mô hình khó tính gradient.



Hình 1.3: Tấn công một pixel trên tập ImageNet

- **Khả năng đánh lừa:** Để đảm bảo ảnh đối kháng không bị phát hiện bằng mắt thường, các phương pháp tấn công đối kháng đều phải giới hạn mức độ thay đổi xuống nhỏ nhất có thể. Thay vì đo lường mức độ thay đổi bằng các hàm phức tạp, phương pháp này giới hạn thay đổi bằng cách giới hạn số lượng pixel được phép thêm nhiều.

Chương 2

Các nghiên cứu liên quan

2.1 Phân loại các phương pháp tấn công đối kháng

Các phương pháp tấn công đối kháng có thể được phân loại theo nhiều tiêu chí khác nhau.

2.1.1 Dựa trên quyền truy cập thông tin

Dựa vào mức độ quyền truy cập vào thông tin của mô hình, các phương pháp tấn công đối kháng được chia thành hai nhóm: tấn công hộp trắng và tấn công hộp đen. Trong tấn công hộp trắng (White-box attack), kẻ tấn công có toàn bộ thông tin về mô hình, bao gồm kiến trúc của mô hình như số lớp, loại lớp, kích thước trọng số, các trọng số và tham số đã được huấn luyện, cũng như gradient của hàm mất mát với đầu vào. Việc có được thông tin đầy đủ này cho phép kẻ tấn công tối ưu hóa trực tiếp để tìm ra nhiều hiệu quả nhất, từ đó tạo ra các tấn công mạnh mẽ nhất. Một số phương pháp phổ biến trong nhóm này là Fast Gradient Sign Method (FGSM) [7], sử dụng gradient để tạo nhiễu theo hướng làm tăng tổn thất, Projected Gradient Descent (PGD) [10], mở rộng FGSM với các bước lặp gradient nhỏ hơn, và Carlini & Wagner (C&W) Attack [1], dựa trên tối ưu hóa để tìm nhiễu nhỏ và hiệu quả nhất.

Ngược lại, trong tấn công hộp đen (black-box attack), kẻ tấn công không có bất kỳ thông tin nào về kiến trúc hoặc gradient của mô hình mà chỉ dựa trên đầu ra của mô hình, chẳng hạn như xác suất nhãn hoặc nhãn dự đoán, và khả năng thử nghiệm, đánh giá mô hình với các đầu vào đã được thay đổi. Các tấn công hộp đen thường sử dụng các kỹ thuật tối ưu hóa hoặc tìm kiếm để ước lượng gradient một cách gián tiếp. Ví dụ, One-Pixel Attack sử dụng tối ưu hóa vi phân để xác định pixel cần thay đổi nhằm làm sai lệch kết quả, trong khi Zeroth Order Optimization (ZOO) Attack ước lượng gradient bằng cách thay đổi nhỏ đầu vào và quan sát đầu ra tương ứng. Mặc dù tấn công hộp đen thường kém hiệu quả hơn so với tấn công hộp trắng, nhưng chúng phù hợp với các kịch bản thực tế khi thông tin về mô hình không sẵn có.

2.1.2 Dựa trên mục tiêu tấn công

Dựa trên mục tiêu tấn công, các phương pháp đối kháng được chia thành tấn công có mục tiêu (Targeted attacks) và tấn công không mục tiêu (Non-targeted attacks). Trong

tấn công có mục tiêu, kẻ tấn công cố gắng thay đổi dự đoán của mô hình thành một nhãn cụ thể bằng cách tối đa hóa xác suất của nhãn mục tiêu trong đầu ra của mô hình. Một số phương pháp điển hình trong nhóm này bao gồm Carlini & Wagner (C&W) Attack, sử dụng tối ưu hóa để đạt nhãn mục tiêu với nhiễu nhỏ nhất, và Zeroth Order Optimization (ZOO) Attack [2], hỗ trợ các tấn công hộp đen bằng cách tối đa hóa xác suất của nhãn mục tiêu mà không yêu cầu thông tin gradient. Tấn công có mục tiêu thường khó thực hiện hơn vì cần tối ưu hóa mạnh để đạt được nhãn mục tiêu cụ thể.

Trong khi đó, tấn công không mục tiêu chỉ nhằm làm cho mô hình phân loại sai nhãn ban đầu mà không quan tâm đến lớp đích cụ thể. Các phương pháp này tập trung vào việc giảm xác suất của nhãn đúng, đẩy đầu vào ra khỏi quyết định phân loại ban đầu. Ví dụ nổi bật bao gồm Fast Gradient Sign Method (FGSM), sử dụng gradient để nhanh chóng làm sai lệch nhãn mà không hướng tới một lớp cụ thể, Projected Gradient Descent (PGD), lặp lại các bước gradient nhỏ để đạt hiệu quả cao hơn FGSM, và One-Pixel Attack, chỉ thay đổi một pixel nhưng đủ để làm sai nhãn, thích hợp cho các tấn công hộp đen. Tấn công không mục tiêu dễ thực hiện hơn và thường có tỷ lệ thành công cao hơn trong thực tế.

2.1.3 Dựa trên loại nhiễu thêm vào

Cuối cùng, các tấn công đối kháng có thể được phân loại dựa trên loại nhiễu được thêm vào đầu vào, bao gồm nhiễu L_0 , nhiễu L_2 , và nhiễu L_∞ .

Đối với nhiễu L_0 , số lượng pixel được thay đổi trong đầu vào bị giới hạn, tập trung vào việc thay đổi ít pixel nhất có thể để nhiễu khó phát hiện hơn. Một ví dụ tiêu biểu cho loại này là One-Pixel Attack, trong đó chỉ thay đổi một pixel duy nhất thông qua thuật toán tối ưu hóa nhằm xác định pixel quan trọng nhất. Loại nhiễu này phù hợp khi yêu cầu tính ẩn cao, tuy nhiên, nó đòi hỏi thời gian tính toán lớn.

Nhiễu L_2 giới hạn tổng cường độ thay đổi trong đầu vào, được đo bằng khoảng cách Euclidean. Phương pháp này ưu tiên giữ cho đầu vào bị tấn công trông tự nhiên và gần giống với đầu vào gốc. Một ví dụ phổ biến là Carlini & Wagner (C&W) Attack, sử dụng tối ưu hóa để tìm ra nhiễu nhỏ nhất nhưng vẫn hiệu quả. Nhiễu L_2 đặc biệt phù hợp khi yêu cầu tính tự nhiên của đầu vào đối kháng.

Cuối cùng, nhiễu L_∞ giới hạn mức thay đổi lớn nhất của một pixel trong đầu vào và tập trung vào việc kiểm soát độ lớn của từng thay đổi để đảm bảo nhiễu không quá nổi bật. Các phương pháp như Fast Gradient Sign Method (FGSM) và Projected Gradient Descent (PGD) là điển hình trong nhóm này. FGSM thay đổi đầu vào theo hướng gradient với cường độ được kiểm soát bởi tham số ϵ , trong khi PGD mở rộng FGSM bằng cách lặp lại nhiều bước nhỏ hơn để đảm bảo nhiễu luôn nằm trong giới hạn L_∞ -norm. Phương pháp nhiễu L_∞ thường được sử dụng trong các tấn công nhanh và hiệu quả, nhưng nếu ϵ quá lớn, nhiễu có thể dễ bị phát hiện.

2.2 Fast Gradient Sign Method (FGSM)

2.2.1 Mục tiêu

FGSM[7] là một phương pháp tấn công hộp trắng dựa trên việc sử dụng gradient của hàm mất mát để tạo nhiễu đối kháng. Mục tiêu chính của phương pháp này là tìm một nhiễu η nhỏ nhất để làm tăng tổn thất của mô hình $J(\theta, x, y)$, từ đó làm sai lệch nhãn dự đoán. Nhiễu này được tính bằng cách điều chỉnh đầu vào theo hướng gradient của hàm mất mát, giúp đẩy đầu vào ra khỏi biên quyết định của mô hình. Cụ thể, nhiễu được xác định thông qua công thức $\eta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$, trong đó ϵ là hệ số kiểm soát độ lớn của nhiễu, $\nabla_x J(\theta, x, y)$ là gradient của hàm mất mát theo đầu vào x , và sign là hàm lấy dấu của gradient để đảm bảo nhiễu có hướng thay đổi nhất quán. Kết quả của quá trình này là một đầu vào bị tấn công x' , được tính bởi $x' = x + \eta$. Đầu vào mới x' sẽ rất gần với x theo cảm nhận của con người nhưng đủ để khiến mô hình phân loại sai.

2.2.2 Ưu - nhược điểm

Phương pháp FGSM có nhiều ưu điểm đáng chú ý. Thứ nhất, đây là một phương pháp đơn giản và nhanh chóng do chỉ yêu cầu tính gradient một lần duy nhất mà không cần lặp lại các bước tính toán phức tạp. Điều này giúp FGSM phù hợp với các hệ thống lớn và tốn ít tài nguyên. Thứ hai, FGSM rất hiệu quả trong việc đánh giá tính mạnh mẽ của mô hình, đóng vai trò như một phương pháp nền tảng để thử nghiệm độ nhạy của mô hình trước các nhiễu nhỏ. Tuy nhiên, FGSM cũng có những nhược điểm. Hiệu quả của FGSM thấp hơn so với các phương pháp tấn công lặp như PGD hay C&W do chỉ thực hiện một bước gradient, khiến nó không khai thác triệt để các điểm yếu của mô hình. Ngoài ra, các phương pháp phòng thủ như Adversarial Training có thể làm giảm hiệu quả của FGSM, vì nhiễu tạo ra khá đơn giản và dễ bị phát hiện. Cuối cùng, kết quả của FGSM phụ thuộc rất lớn vào giá trị ϵ . Nếu ϵ quá nhỏ, nhiễu sẽ không đủ mạnh để làm sai nhãn, trong khi nếu ϵ quá lớn, nhiễu sẽ trở nên dễ nhận biết bởi con người và mất đi tính ẩn.

2.2.3 Ứng dụng

FGSM được sử dụng rộng rãi trong các nghiên cứu và ứng dụng thực tế. Một trong những ứng dụng chính của FGSM là kiểm tra tính dễ bị tổn thương của các mô hình học sâu trước các tấn công đối kháng. Ngoài ra, FGSM còn đóng vai trò quan trọng trong việc đào tạo mô hình mạnh mẽ hơn thông qua Adversarial Training, nơi nhiễu FGSM được đưa vào quá trình huấn luyện để giúp mô hình học cách chống chịu các tấn công. Hơn nữa, FGSM cũng được xem là cơ sở cho các phương pháp tấn công phức tạp hơn như PGD, giúp mở rộng khả năng tấn công sang nhiều kịch bản thực tế.

2.3 Projected Gradient Descent (PGD)

2.3.1 Mục tiêu

PGD[10] là một phương pháp tấn công lặp lại mở rộng từ FGSM, với mục tiêu tối đa hóa tổn thất của mô hình $J(\theta, x, y)$ thông qua nhiều bước gradient nhỏ. Phương pháp này có thể được sử dụng để làm sai nhãn dự đoán hoặc hướng đầu vào về một nhãn mục tiêu cụ thể. Quá trình tấn công bằng PGD bao gồm việc tính gradient của hàm mất mát $J(\theta, x, y)$ tại mỗi bước để điều chỉnh đầu vào và thêm nhiễu đối kháng được xác định bởi $x^{(t+1)} = \text{Proj}_{\mathcal{B}(x, \epsilon)}(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^{(t)}, y)))$. Ở đây, α là kích thước bước gradient tại mỗi lần lặp, $\mathcal{B}(x, \epsilon)$ là miền nhiễu được giới hạn bởi chuẩn L_p (ví dụ: L_∞ -norm với bán kính ϵ), và Proj là phép dự phóng để đảm bảo rằng $x^{(t+1)}$ nằm trong miền cho phép. Quá trình này được lặp lại nhiều lần cho đến khi đạt số bước tối đa hoặc đạt hiệu quả tấn công mong muốn. PGD được xem là một trong những phương pháp tấn công mạnh mẽ nhất và thường được sử dụng để đánh giá độ mạnh mẽ của mô hình học sâu trước các tấn công đối kháng.

2.3.2 Ưu - nhược điểm

PGD mang lại nhiều lợi thế nổi bật. So với FGSM, PGD thực hiện nhiều bước gradient nhỏ, từ đó giúp khai thác triệt để các điểm yếu của mô hình và đạt hiệu quả tấn công cao hơn. Bên cạnh đó, PGD rất linh hoạt và có thể áp dụng cho cả tấn công có mục tiêu và không mục tiêu. Đây cũng là phương pháp được coi là chuẩn mực để đánh giá tính mạnh mẽ của mô hình học sâu trước các tấn công đối kháng. Tuy nhiên, PGD cũng có nhược điểm. Phương pháp này tốn nhiều thời gian và tài nguyên tính toán hơn so với FGSM do phải thực hiện nhiều bước lặp. Hơn nữa, hiệu quả của PGD có thể giảm đáng kể khi đối mặt với các mô hình được bảo vệ bằng Adversarial Training. Ngoài ra, kết quả của PGD phụ thuộc vào các tham số như kích thước bước α , số bước lặp và miền nhiễu ϵ , đòi hỏi sự tinh chỉnh phù hợp.

2.3.3 Ứng dụng

PGD được sử dụng phổ biến để đánh giá độ mạnh mẽ của các mô hình học sâu trước các tấn công đối kháng. Ngoài ra, PGD còn được áp dụng trong việc xây dựng các mô hình mạnh mẽ hơn thông qua Adversarial Training, nơi phương pháp này được dùng để tạo nhiễu trong quá trình huấn luyện. PGD cũng được sử dụng để kiểm tra khả năng chống chịu của các hệ thống học máy trong nhiều ứng dụng thực tế như nhận diện hình ảnh, xử lý ngôn ngữ tự nhiên và an ninh mạng.

2.4 Carlini & Wagner (C&W) Attack

2.4.1 Mục tiêu

Mục tiêu chính của Carlini & Wagner (C&W) là một phương pháp tấn công hợp pháp dựa trên tối ưu hóa. Phương pháp này nhằm tạo ra các nhiễu nhỏ nhất có thể để làm sai lệch nhãn dự đoán của mô hình, nhưng vẫn giữ cho đầu vào bị tấn công trông

tự nhiên. Đồng thời, C&W [1] tối thiểu hóa khoảng cách giữa đầu vào gốc và đầu vào bị tấn công, thường sử dụng chuẩn L_2 . Mục tiêu thứ ba của C&W là tối đa hóa khả năng thay đổi nhãn dự đoán bằng cách tối ưu hóa hàm mất mát đặc biệt:

$$\text{minimize } \|x' - x\|_p + c \cdot f(x')$$

Trong đó, $\|x' - x\|_p$ là khoảng cách giữa đầu vào gốc và đầu vào bị tấn công, $f(x')$ là hàm mất mát đo lường mức độ phân loại sai của đầu vào x' , và c là tham số điều chỉnh giữa hai mục tiêu. C&W hỗ trợ cả tấn công có mục tiêu (targeted) và không mục tiêu (non-targeted), phụ thuộc vào cách thiết lập hàm mất mát.

2.4.2 Ưu - nhược điểm

Một trong những ưu điểm của C&W là khả năng tạo ra nhiễu tinh vi, khó bị phát hiện bằng mắt thường, giúp nó vượt qua nhiều biện pháp phòng thủ. Nó cũng rất hiệu quả với các mô hình phòng thủ mạnh như Adversarial Training. C&W linh hoạt, hỗ trợ cả tấn công có mục tiêu và không mục tiêu, và có thể sử dụng các chuẩn L_0 , L_2 , hoặc L_∞ . Tuy nhiên, phương pháp này cũng có nhược điểm. Việc tối ưu hóa phức tạp của C&W dẫn đến chi phí tính toán cao hơn so với các phương pháp đơn giản như FGSM hoặc PGD. Nó cũng không khả dụng trong các kịch bản hộp đen vì phụ thuộc vào gradient của mô hình.

2.5 Zero-Order Optimization (ZOO) Attack

2.5.1 Mục tiêu

ZOO Attack [2] là một phương pháp tấn công hộp đen dựa trên tối ưu hóa bậc 0 (zero-order optimization). Mục tiêu chính của ZOO là tạo ra nhiễu đối kháng mà không yêu cầu thông tin về gradient hoặc kiến trúc của mô hình. Phương pháp này ước lượng gradient thông qua thay đổi nhỏ trong đầu vào và quan sát sự thay đổi trong đầu ra:

$$\nabla_x J(x) \approx \frac{J(x + \delta e_i) - J(x)}{\delta}$$

Trong đó δ là giá trị thay đổi nhỏ để ước lượng gradient, và e_i là vector đơn vị theo hướng pixel i . ZOO tối ưu hóa để làm sai nhãn dự đoán hoặc hướng đầu vào về một nhãn mục tiêu.

2.5.2 Ưu - nhược điểm

Một ưu điểm của ZOO [2] là nó phù hợp cho các mô hình hộp đen vì không yêu cầu thông tin về gradient hoặc kiến trúc của mô hình. Nó linh hoạt và hỗ trợ cả tấn công có mục tiêu và không mục tiêu. ZOO cũng hiệu quả với các mô hình không khả vi, khi không thể trực tiếp tính toán gradient. Tuy nhiên, ZOO cũng có nhược điểm. Quá trình ước lượng gradient đòi hỏi nhiều lần đánh giá mô hình, dẫn đến chi phí tính toán cao. Nếu số lượng đánh giá mô hình bị giới hạn, việc ước lượng gradient có thể không chính xác, làm giảm hiệu quả của nhiễu.

Chương 3

Tấn công một pixel

3.1 Mô hình hóa bài toán

Tấn công đối kháng có thể coi là bài toán tối ưu hóa có điều kiện. Ta biểu diễn mỗi ảnh dưới dạng một vector trong đó mỗi phần tử đại diện cho một điểm ảnh. Gọi f là mô hình phân loại ảnh cần tấn công nhận đầu vào n chiều, $\mathbf{x} = (x_1, x_2, \dots, x_n)$ là ảnh ban đầu có nhãn đúng là t . Xác suất $\mathbf{x}$ mang nhãn t là $f_t(\mathbf{x})$. Vector $e(\mathbf{x}) = (e_1, \dots, e_n)$ là lượng nhiễu đối kháng được thêm vào $\mathbf{x}$, adv là nhãn mục tiêu, có nghĩa là ta mong muốn làm cho mô hình nhận diện $\mathbf{x}$ thuộc nhãn adv . L là lượng thay đổi tối đa được thực hiện. L được đo bằng độ dài của vector $e(\mathbf{x})$. Mục tiêu của tấn công đối kháng có mục tiêu là tìm lời giải tối ưu $e(\mathbf{x})^*$ cho bài toán sau:

$$\begin{aligned} \max_{e(\mathbf{x})^*} \quad & f_{adv}(\mathbf{x} + e(\mathbf{x})) \\ \text{subject to} \quad & \|e(\mathbf{x})\| \leq L \end{aligned}$$

Để giải bài toán này, ta cần xác định được vị trí cần bị thay đổi và mức độ thay đổi của mỗi vị trí. Đối với phương pháp tấn công một pixel, điều kiện của bài toán hơi khác một chút:

$$\begin{aligned} \max_{e(\mathbf{x})^*} \quad & f_{adv}(\mathbf{x} + e(\mathbf{x})) \\ \text{subject to} \quad & \|e(\mathbf{x})\|_0 \leq d \end{aligned}$$

trong đó d là một số nhỏ. Trong trường hợp tấn công một pixel thì $d = 1$. Các phương pháp khác thường thay đổi khá nhiều chiều của ảnh, trong khi phương pháp này chỉ thay đổi d chiều.

Các ảnh đối kháng thường được tạo ra bằng cách làm nhiễu tất cả điểm ảnh với một giới hạn đối với tổng mức độ nhiễu, trong khi tấn công một vài điểm ảnh trong bài báo này làm ngược lại, có nghĩa là chỉ tập trung thay đổi vài pixel những không giới hạn mức độ thay đổi.

3.2 Tiến hóa vi phân

Giải thuật tiến hóa là một thuật toán tối ưu hóa heuristic lấy ý tưởng từ cơ chế tiến hóa trong tự nhiên như biến dị, tái tổ hợp và chọn lọc tự nhiên. Mục tiêu của giải thuật

tiến hóa là tìm cấu hình tối ưu cho một hệ thống quần thể với các ràng buộc cụ thể [14] [4].

Tiến hóa vi phân là một nhánh nhỏ của giải thuật tiến hóa. Tiến hóa vi phân được phát triển bởi Storn và Price. Đây là một giải thuật hiệu quả để áp dụng cho các bài toán tối ưu hóa không khả vi, hàm mục tiêu phức tạp, không tuyến tính. Thuật toán này được kì vọng là tìm được lời giải tối ưu có chất lượng cao hơn các phương pháp gradient hay các phương pháp khác thuộc giải thuật tiến hóa[3].

3.2.1 Nguyên lý hoạt động của tiến hóa vi phân

Thuật toán tiến hóa vi phân áp dụng trong phương pháp tấn công một pixel có ba quá trình quan trọng sau:

1. **Khởi tạo:** Khởi tạo ngẫu nhiên quần thể các cá thể ban đầu. Trong phương pháp tấn công một pixel này, các cá thể là các bộ nhiễu.
2. **Tạo đột biến:** Ở mỗi lần lặp, một bộ nhiễu con sẽ được tạo ra dựa trên các cá thể cha mẹ trong quần thể ban đầu thông qua các phép đột biến và tái tổ hợp.
3. **Chọn lọc:** Cá thể con sẽ được giữ lại nếu chúng có khả năng thích nghi cao hơn so với cá thể cha mẹ (tức là giá trị hàm mục tiêu của chúng tốt hơn so với thể hệ cha mẹ).

3.2.2 Một vài ưu điểm của tiến hóa vi phân

Tiến hóa vi phân là một giải thuật mạnh mẽ và được áp dụng để giải quyết nhiều bài toán tối ưu hiện nay. Phương pháp này không sử dụng thông tin gradient để hoạt động do đó nó áp dụng được cho nhiều bài toán tối ưu mà hàm mục tiêu không khả vi hoặc không được biết trước. Việc sử dụng tiến hóa vi phân để tạo ra các hình ảnh mang nhiều đối kháng đem lại một vài ưu điểm chính sau đây:

Khả năng tìm được cực trị cao hơn: Tiến hóa vi phân ít bị ảnh hưởng bởi các cực trị địa phương hơn so với các phương pháp cổ điển như gradient descent hay các giải thuật tham lam khác. Bằng cách kết hợp linh hoạt cơ chế chọn lọc tự nhiên và cơ chế biến dị, tái tổ hợp giúp giải thuật tiến hóa vi phân có khả năng giữ được sự đa dạng của quần thể đồng thời mở rộng được không gian tìm kiếm giúp tăng khả năng tìm được cực trị toàn cục cho các bài toán tối ưu.

Cần ít thông tin từ hệ thống mục tiêu: Giải thuật tiến hóa vi phân có thể áp dụng cho nhiều bài toán tối ưu với các hàm mục tiêu phức tạp mà không yêu cầu bài toán phải khả vi như các phương pháp khác. Đây là một yếu tố rất quan trọng trong bài toán tấn công đối kháng do việc có được thông tin về mô hình mục tiêu là không thực tế và rất nhiều mô hình không khả vi [17].

Đơn giản và linh hoạt: Phương pháp này không phụ thuộc vào kiến trúc mô hình, do đó có thể sử dụng để kiểm thử hoặc tấn công cho nhiều mô hình học sâu khác nhau. Để có thể thực hiện được các cuộc tấn công, chỉ cần biết kết quả nhãn mà mô hình đưa ra mà không cần thêm các thông tin khác về mô hình mục tiêu.

3.3 Tấn công một pixel

Giải thuật tiến hóa vi phân được sử dụng để tìm kiếm pixel cần chỉnh sửa trong ảnh gốc từ đó tạo ra ảnh đối kháng. Các bước cài đặt thuật toán trong bài toán tấn công một pixel như sau:

Khởi tạo: Điểm ảnh thay đổi được chọn ngẫu nhiên theo phân phối đều. Việc lựa chọn điểm ảnh theo phân phối đều đảm bảo tất cả điểm ảnh đều có vai trò tương đương nhau, đảm bảo độ đa dạng cho quần thể khởi tạo. Các giá trị màu r, g, b được khởi tạo trong khoảng từ $[0, 255]$. Ở đây, các tác giả sử dụng phân phối chuẩn với $\mu = 128$, độ lệch chuẩn là $\sigma = 127$.

Tạo đột biến: Một đột biến $V(i)$ là đột biến tạo ra cho giải pháp thứ i bằng cách chọn ngẫu nhiên 3 cá thể trong quần thể kết với hệ số tỉ lệ:

$$V_i = x_{r1}(g) + F(x_{r2}(g) - x_{r3}(g))$$

Trong đó:

- $r1, r2, r3$ là các giải pháp khác nhau được lấy ngẫu nhiên trong quần thể.
- F là hệ số tỉ lệ thường được lấy trong khoảng $[0, 2]$, trong quá trình thực nghiệm, giá trị F được lấy là 0.5.

Lưu ý rằng, chúng ta hoàn toàn có thể lựa chọn cách tạo đột biến khác. Chúng em đã thay đổi cách tạo đột biến của bài này thành trung bình giá trị của $x_{r1}(g), x_{r2}(g), x_{r3}(g)$ và kết quả sẽ được trình bày chi tiết ở phần thực nghiệm.

Tổ hợp trong tiến hóa vi phân: Thế hệ con $U(i)$ là cá thể con được tạo ra từ thế hệ cha $X(i)$ và đột biến $V(i)$ bằng cách sử dụng giá trị ngẫu nhiên kết hợp tỉ lệ tổ hợp cho trước:

$$U_i[j] = \begin{cases} V_i[j] & \text{khi } r \leq CR \\ X_i[j] & \text{khi } r > CR \end{cases}$$

Trong đó:

- CR là tỉ lệ tổ hợp được lấy trong đoạn từ $[0, 1]$.
- r là giá trị ngẫu nhiên được tạo ra.

Để tạo ra thế hệ con U, chúng ta sẽ lấy ngẫu nhiên một số r trong đoạn từ $[0, 1]$ và lấy giá trị đó so sánh với giá trị tổ hợp được định nghĩa trước theo công thức trên.

Nếu tỉ lệ tổ hợp CR quá lớn thì khả năng lấy các đặc điểm của đột biến sẽ lớn hơn điều này có thể tạo ra nhiều cá thể mới, giúp mở rộng không gian tìm kiếm tuy nhiên có thể làm mất đi những đặc điểm tốt từ thế hệ trước.

Nếu tỉ lệ tổ hợp CR quá nhỏ thì khả năng lấy được điểm của thể hệ trước sẽ lớn hơn điều này có thể khiến quần thể ổn định qua các thế hệ tuy nhiên điều này cũng làm giảm không gian tìm kiếm và có thể khiến thuật toán dễ bị sa lầy vào cực trị địa phương ảnh hưởng đến độ hiệu quả của thuật toán.

Chọn lọc trong tiến hóa vi phân: Nhiều con U và nhiều cha X sẽ được đưa vào mô hình để dự đoán, nếu nhiều con U cho ra kết quả hàm mục tiêu cao hơn so với thể hệ cha X của chúng thì nó sẽ được giữ lại, loại bỏ thể hệ cha X và ngược lại thì thể hệ cha X sẽ được giữ lại.

Các bước trên sẽ được lặp lại cho đến khi thuật toán kết thúc.

Điều kiện dừng của thuật toán:

Khi chạy hết số vòng lặp quy định: Nếu số vòng lặp quy định quá nhỏ thì có thể chưa tìm được nghiệm tối ưu thì thuật toán đã phải dừng lại và ngược lại nếu số vòng lặp quá lớn sẽ làm tăng độ phức tạp và thời gian chạy của thuật toán.

Khi đã điều kiện về xác suất nhận đầu ra: Nếu tìm được một nhiều đầu tiên thỏa mãn điều kiện về xác suất nhận đầu ra thuật toán sẽ dừng ngay lập tức. Điều kiện về xác suất nhận trong trường hợp tấn công có mục tiêu là $> 90\%$ và trong trường hợp tấn công không mục tiêu là $< 10\%$.

Chương 4

Thực nghiệm và Kết quả

Trong chương này, chúng em sẽ trình bày lại kết quả các thí nghiệm của các tác giả trong bài báo gốc.

4.1 Thiết lập thí nghiệm

Nhóm tác giả thực hiện các thí nghiệm trên hai tập dữ liệu là Kaggle CIFAR10 và ImageNet.

Thí nghiệm trên tập dữ liệu KAGGLE-CIFAR 10

Nhóm tác giả đã thực nghiệm phân loại ảnh có mục tiêu trên ba mạng nơ-ron phổ biến: All convolution network (AllConv) [13], Network in Network (NiN) [9] và VGG16 network [12].

Để tiến hành tấn công vào ba mạng nơ-ron trên, 500 ảnh ngẫu nhiên sẽ được dùng để thực nghiệm, các ảnh này được lấy từ tập dữ liệu KAGGLE-CIFAR 10 (chứa 300 000 ảnh). Với mỗi hình ảnh gốc sẽ có 9 tấn công có mục tiêu, nhằm cố gắng chuyển nhãn kết quả sang 9 nhãn khác với nhãn ban đầu. Kết quả của tấn công không mục tiêu được lấy dựa trên các thí nghiệm tấn công có mục tiêu: Với mỗi ảnh, thực hiện 9 cuộc tấn công có mục tiêu, nếu có 1 cuộc tấn công thành công thì xem như tấn công không có mục tiêu là thành công.

Ngoài ra, nhóm tác giả cũng thực hiện cuộc tấn công với số pixel được thay đổi là 3 và 5. Mục tiêu là để so sánh mức độ hiệu quả giữa tấn công 1 pixel với tấn công 3 pixel và 5 pixel.

Thí nghiệm trên tập dữ liệu ImageNet

Các thực nghiệm phân loại ảnh sử dụng tập dữ liệu ImageNet [5] đều là tấn công không mục tiêu đối với mạng nơ-ron BVLC AlexNet, sử dụng 105 hình ảnh chọn ngẫu nhiên từ ILSVRC 2012. Các thí nghiệm trên ImageNet nhằm kiểm tra xem một thay đổi nhỏ có thể làm nhiễu các mô hình nhận diện với ảnh có kích thước lớn hay không.

4.2 Kết quả thực nghiệm

Dưới đây là tỷ lệ thành công của các cuộc tấn công khác nhau trên hai tập dữ liệu KAGGLE-CIFAR 10 và ImageNet.

	AllConv	NiN	VGG16	BVLC
Targeted	19.82%	23.15%	16.48%	-
Non-targeted	68.71%	71.66%	63.53%	16.04%
Confidence	79.40%	75.02%	67.67%	22.91%

Bảng 4.1: Tỷ lệ thành công khi tấn công một pixel trên 4 mô hình khác nhau. 3 mô hình AllConv, NiN, VGG16 được huấn luyện trên tập CIFAR10, và mô hình BVLC được huấn luyện trên ImageNet.

+ Hàng Confidence của bảng 4.1 là xác suất trung bình của các lớp mục tiêu.

Kết quả cho thấy các cuộc tấn công không có mục tiêu (1 pixel) có tỷ lệ thành công khá cao, lần lượt là 68.71%, 71.66% và 63.53% đối với các mạng nơ-ron trên tập dữ liệu KAGGLE-CIFAR 10, cho thấy tính hiệu quả của phương pháp tấn công này. Từ đó, chúng ta biết rằng các mô hình nhận diện hình ảnh vẫn dễ bị tấn công bởi những thay đổi nhỏ trong hình ảnh đầu vào.

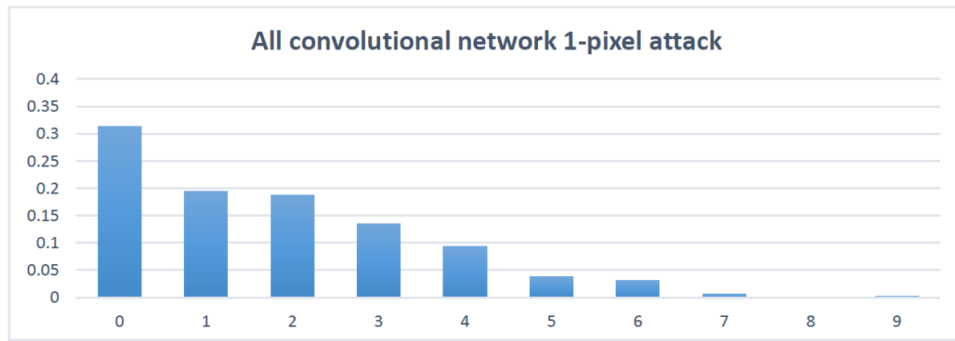
Ngoài ra, các tác giả cũng đưa ra kết quả tấn công 3 pixel (AllConv) và tấn công 5 pixel (NiN):

	3 pixel	5 pixel
Success rate(tar)	40.57%	44.00%
Success rate(non-tar)	86.53%	86.34%
Rate/Labels	79.17%	77.09%

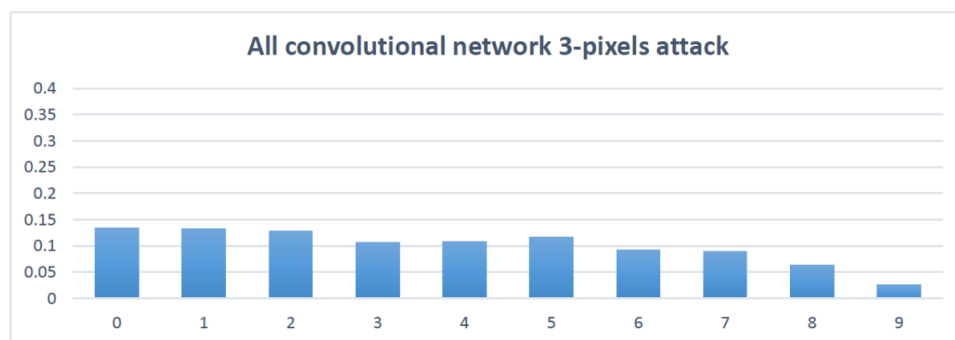
Bảng 4.2: Tỷ lệ thành công khi tấn công 3 pixel trên mạng nơ-ron All convolution network (AllConv) và tấn công 5 pixel trên mạng nơ-ron Network in Network (NiN).

Kết quả từ bảng trên khi so sánh với kết quả tấn công 1 pixel thì tỷ lệ thành công được cải thiện rõ rệt.

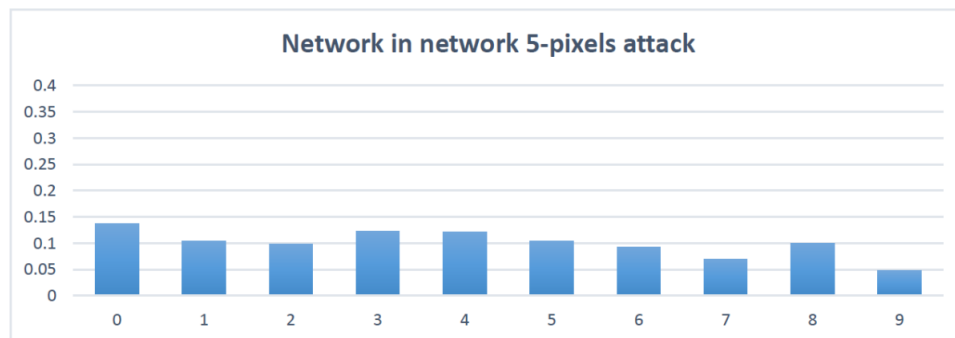
Trung bình khi tấn công 1 pixel, mỗi ảnh có thể bị nhiễu tới hai lớp mục tiêu, nhưng số lượng lớp mục tiêu khi tăng lên 3 pixel và 5 pixel thì rõ ràng số lượng mục tiêu có thể tiếp cận thành công tăng lên đáng kể.



Hình 4.1: Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tấn công 1 pixel.



Hình 4.2: Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tấn công 3 pixel.



Hình 4.3: Biểu đồ thể hiện tỷ lệ ảnh bị làm nhiễu đến một số lượng n nhãn nhất định bằng cách tấn công 5 pixel.

Chi phí để thực hiện cuộc tấn công 1 pixel cũng được các tác giả công bố dựa vào bảng sau:

	AllConv	NiN	VGG16	BVLC
AvgEvaluation	16000	12400	20000	25600
AvgDistortion	123	133	145	158

Bảng 4.3: Đánh giá chi phí thực hiện một cuộc tấn công 1 pixel trên bốn loại mạng nơ-ron AllConv, NiN, VGG16, BVLC.

+ Hàng AvgEvaluation trong bảng 4.3 là số lần đánh giá mà máy tính cần kiểm tra để tạo ra một hình ảnh có thể đánh lừa mạng mô hình nhận diện ảnh.

+ Hàng AvgDistortion trong bảng 4.3 là mức độ thay đổi trong 1 pixel để đánh lừa mô hình nhận diện ảnh.

Chương A

Phụ lục

Trong phần này, nhóm chúng em sẽ trả lời những câu hỏi và thực hiện những gợi ý của thầy thông qua các thực nghiệm do chúng em tự cài đặt để hiểu sâu hơn về phương pháp cũng như cố gắng cải thiện kết quả.

Chúng em đã cài đặt phương pháp Tấn công một pixel bằng ngôn ngữ lập trình Python, và thực hiện tấn công mô hình VGG16 [12] đã được huấn luyện trên tập dữ liệu CIFAR10 [8], với hai kịch bản thí nghiệm như sau.

A.1 Sử dụng phân phối chuẩn ở bước khởi tạo quần thể

Trong bài báo, các tác giả khởi tạo quần thể cho quá trình Tiến hóa vi phân [14] bằng cách chọn ngẫu nhiên điểm ảnh bị thay đổi theo phân phối đều, có nghĩa là xác suất được chọn của mỗi điểm ảnh trên tấm ảnh là như nhau. Và trên điểm ảnh đó, màu sắc mới của điểm ảnh được chọn theo phân phối chuẩn. Chúng em đã thay đổi cách khởi tạo này để thử nghiệm xem kết quả có gì khác biệt không. Cụ thể, thay vì chọn điểm ảnh bị thay đổi theo phân phối đều, chúng em thử chọn điểm đó theo phân phối chuẩn.

Trong thuật toán Tiến hóa vi phân [14], ở mỗi vòng lặp, mỗi cá thể đột biến (ở đây là một bộ nhiễu) được tạo ra từ 3 cá thể ngẫu nhiên trong quần thể. Bộ nhiễu mới này có thể có các giá trị màu hoặc tọa độ điểm ảnh vượt quá giới hạn cho phép, vì vậy sau khi tạo đột biến cần có phương pháp đảm bảo các giá trị này nằm trong giới hạn của chúng.

- Phương pháp thứ nhất là **clip**: có nghĩa là với các giá trị nhỏ hơn giới hạn thấp nhất thì đưa nó về bằng giá trị nhỏ nhất và ngược lại.
- Phương pháp thứ hai là **resample**: các giá trị vượt ra ngoài khoảng cho phép được thay bằng một giá trị ngẫu nhiên trong khoảng cho phép.

Chúng em đã thực hiện tấn công không mục tiêu trên mô hình VGG16 [12] với hai cách khởi tạo quần thể và hai chiến thuật xử lý giá trị ngoài phạm vi như trên. Các ảnh dùng để tấn công là 100 ảnh đầu tiên của tập dữ liệu CIFAR10. Kết quả được ghi lại trong bảng sau:

Cách chọn điểm ảnh bị thay đổi	Xử lý giá trị ngoài phạm vi	Tỉ lệ tấn công thành công
Phân phối đều	Clip	64%
Phân phối đều	Resample	62%
Phân phối chuẩn	Clip	66%
Phân phối chuẩn	Resample	61%

Bảng A.1: Kết quả thí nghiệm so sánh hai cách chọn điểm ảnh bị thay đổi

A.2 Tạo đột biến theo trung bình cộng của ba cá thể

Trong thuật toán tiến hóa vi phân [14], mỗi đột biến được tạo từ ba cá thể trong quần thể với công thức như sau:

$$x_i(g+1) = x_{r1}(g) + F(x_{r2}(g) - x_{r3}(g)), \quad r1 \neq r2 \neq r3,$$

Chúng em đã thử thay đổi bước này bằng cách lấy đột biến bằng trung bình của ba cá thể:

$$x_i(g+1) = \frac{x_{r1}(g) + x_{r2}(g) + x_{r3}(g)}{3}, \quad r1 \neq r2 \neq r3,$$

Với mỗi cách tạo đột biến trên, nhóm cũng dùng 2 cách xử lý giá trị ngoài phạm vi khác nhau là clip và resample. Kết quả thu được như sau:

Cách tạo đột biến	Chọn điểm ảnh bị thay đổi	Xử lý giá trị ngoài phạm vi	Tỉ lệ tấn công thành công
Như bài báo gốc	Phân phối đều	Clip	64%
Như bài báo gốc	Phân phối đều	Resample	62%
Lấy trung bình	Phân phối đều	Clip	42%
Lấy trung bình	Phân phối đều	Resample	40%

Bảng A.2: Kết quả thí nghiệm so sánh hai cách tạo đột biến

A.3 So sánh với việc thử thay đổi từng pixel

Các ảnh cần tạo nhiễu thuộc tập CIFAR10, đều có kích thước 32 x 32 điểm ảnh. Nếu ta thực hiện tấn công theo phương pháp brute force, cụ thể là đổi màu từng điểm ảnh về trắng hoặc đen, thì số vòng lặp có thể lên đến $32 \times 32 \times 2 = 2048$.

Trong các thí nghiệm tấn công một pixel trên, số vòng lặp tối đa được đặt là 100. Như vậy trong trường hợp tệ nhất, phương pháp này vẫn chạy nhanh hơn phương pháp brute force hơn 20 lần. Và với thời gian chạy như vậy, tỉ lệ tấn công thành công đã đạt mức rất cao, trên 60%.

Việc tấn công thành công hay không phụ thuộc rất nhiều vào hình ảnh mục tiêu, chứ không chỉ là vấn đề của phương pháp. Với giới hạn là chỉ một pixel được thay đổi,

có những ảnh gần như không có cách thêm nhiễu để đánh lừa thành công mô hình. Khi tăng số vòng lặp của tiến hóa vi phân [14] lên 300 hoặc 400, tỉ lệ tấn công thành công không tăng lên nhiều. Vì vậy số vòng lặp tối đa là 100 đã đủ tốt cho phương pháp tấn công một pixel này.

Mặt khác, việc brute force qua từng điểm ảnh là rất tốn thời gian (gấp khoảng 20 lần) trong khi không đạt được hiệu quả cao hơn do giới hạn đã đặt ra của bài toán.

A.4 Nhận xét

Qua kết quả của hai thí nghiệm, ta có thể thấy giữa hai phương pháp xử lý giá trị ngoài phạm vi, kết quả từ phương pháp clip luôn luôn tốt hơn phương pháp resample một chút. Điều này có thể là do phương pháp clip gây ra ít thay đổi với giá trị ngoài phạm vi hơn, giúp giữ sự ổn định nhất định của mỗi lần đột biến.

Hai cách chọn điểm ảnh bị thay đổi là phân phối đều và phân phối chuẩn cho ra kết quả chênh lệch không đáng kể. Trong khi đó hai cách tạo đột biến có kết quả khác biệt hoàn toàn. Lấy đột biến bằng trung bình của ba cá thể cho kết quả tệ hơn khá nhiều so với phương pháp Tiến hóa vi phân thông thường [14].

Tài liệu tham khảo

- [1] N. Carlini and D. Wagner, “Towards evaluating the robustness of neural networks,” in *2017 IEEE Symposium on Security and Privacy (SP)*. Ieee, 2017, pp. 39–57.
- [2] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, “Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models,” in *Proceedings of the 10th ACM workshop on artificial intelligence and security*, 2017, pp. 15–26.
- [3] P. Civicioglu and E. Besdok, “A conceptual comparison of the cuckoo-search, particle swarm optimization, differential evolution and artificial bee colony algorithms,” *Artificial intelligence review*, vol. 39, pp. 315–346, 2013.
- [4] S. Das and P. N. Suganthan, “Differential evolution: A survey of the state-of-the-art,” *IEEE transactions on evolutionary computation*, vol. 15, no. 1, pp. 4–31, 2010.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [6] A. Fawzi, S.-M. Moosavi-Dezfooli, and P. Frossard, “The robustness of deep networks: A geometrical perspective,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 50–62, 2017.
- [7] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *arXiv preprint arXiv:1412.6572*, 2014.
- [8] A. Krizhevsky, G. Hinton *et al.*, “Learning multiple layers of features from tiny images,” 2009.
- [9] M. Lin, “Network in network,” *arXiv preprint arXiv:1312.4400*, 2013.
- [10] A. Madry, “Towards deep learning models resistant to adversarial attacks,” *arXiv preprint arXiv:1706.06083*, 2017.
- [11] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, “Deepfool: a simple and accurate method to fool deep neural networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2574–2582.
- [12] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

- [13] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, “Striving for simplicity: The all convolutional net,” *arXiv preprint arXiv:1412.6806*, 2014.
- [14] R. Storn and K. Price, “Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces,” *Journal of global optimization*, vol. 11, pp. 341–359, 1997.
- [15] J. Su, D. V. Vargas, and K. Sakurai, “One pixel attack for fooling deep neural networks,” *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019.
- [16] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, “Deepface: Closing the gap to human-level performance in face verification,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 1701–1708.
- [17] D. V. Vargas and J. Murata, “Spectrum-diverse neuroevolution with unified neural models,” *IEEE transactions on neural networks and learning systems*, vol. 28, no. 8, pp. 1759–1773, 2016.